



GyaanVault

SPPU M.Sc. Data Science — Free Study Material

Lab: Statistics for Data Science

DS-504-MJP

Solved Practical Manual

Semester I · Lab (MJP)

This solved practical manual covers the Statistics for Data Science lab assignments from the official SPPU M.Sc. Data Science syllabus. Each practical states the aim followed by a complete, runnable solution and the expected output. Use it as a reference — type the code yourself and experiment with the parameters to learn effectively.

Requirements: Python 3.x with numpy, pandas, matplotlib, scikit-learn, scipy installed (pip install numpy pandas matplotlib scikit-learn scipy).

Practical 1: Diagrammatic representation & descriptive statistics

Aim: Draw bar, line and pie charts and compute basic descriptive statistics for raw data.

```
import numpy as np
import matplotlib.pyplot as plt

subjects = ["Stats", "Maths", "DS", "AI", "RM"]
marks    = [72, 65, 80, 58, 90]

fig, ax = plt.subplots(1, 3, figsize=(14, 4))
ax[0].bar(subjects, marks, color="#3b82f6"); ax[0].set_title("Bar chart")
ax[1].plot(subjects, marks, marker="o");      ax[1].set_title("Line chart")
ax[2].pie(marks, labels=subjects, autopct="%1.1f%%"); ax[2].set_title("Pie chart")
plt.tight_layout(); plt.show()

print("Mean :", np.mean(marks))
print("Range:", np.max(marks) - np.min(marks))
```

Practical 2: Measures of central tendency, dispersion, skewness & kurtosis

Aim: Summarise a dataset using mean/median/mode, variance, standard deviation, skewness and kurtosis.

```
import pandas as pd
from scipy import stats

data = pd.Series([12, 15, 15, 18, 20, 21, 21, 21, 25, 30])
print("Mean      :", data.mean())
print("Median    :", data.median())
print("Mode      :", data.mode().tolist())
print("Variance   :", data.var())
print("Std deviation :", data.std())
print("Skewness   :", stats.skew(data))
print("Kurtosis   :", stats.kurtosis(data))
```

Practical 3: Correlation and linear regression

Aim: Measure correlation and fit a simple linear regression line $Y = a + bX$.

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([1,2,3,4,5,6,7,8]).reshape(-1,1)
y = np.array([2,4,5,4,5,7,8,9])
```

```
print("Correlation r:", np.corrcoef(X.ravel(), y)[0,1])
model = LinearRegression().fit(X, y)
print("Intercept a:", model.intercept_)
print("Slope b      :", model.coef_[0])
print("R-squared   :", model.score(X, y))
```

Practical 4: Probability — simple, conditional, Bayes' theorem

Aim: Solve probability problems including conditional probability and Bayes' theorem.

```
# Bayes' theorem: P(Disease | Positive)
p_disease = 0.01          # prior
p_pos_given_disease = 0.99 # sensitivity
p_pos_given_healthy = 0.05 # false-positive rate

p_pos = p_pos_given_disease * p_disease + p_pos_given_healthy * (1 - p_disease)
p_disease_given_pos = (p_pos_given_disease * p_disease) / p_pos
print("P(Disease | Positive) =", round(p_disease_given_pos, 4))
```

Practical 5: Logistic regression

Aim: Build a logistic regression classifier and evaluate its accuracy.

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=1)
clf = LogisticRegression(max_iter=5000).fit(Xtr, ytr)
print("Accuracy:", accuracy_score(yte, clf.predict(Xte)))
```

Practical 6: Hypothesis testing — t-test, Chi-square, ANOVA, F-test

Aim: Perform common statistical tests and interpret the p-values.

```
from scipy import stats
import numpy as np

a = [23, 25, 21, 22, 27]; b = [30, 31, 29, 35, 33]
print("t-test   :", stats.ttest_ind(a, b))
print("ANOVA    :", stats.f_oneway(a, b, [26, 24, 28, 25, 27]))

obs = np.array([[20, 30], [25, 25]])
print("Chi-sq   :", stats.chi2_contingency(obs)[:2])
```

Interpretation: if p-value < 0.05 the result is statistically significant — reject the null hypothesis.

Practical 7: Case study on a real-world dataset

Aim: Apply all the above techniques end-to-end on a real dataset (e.g. a Kaggle CSV).

```
import pandas as pd
df = pd.read_csv("your_dataset.csv") # e.g. from kaggle.com
print(df.describe())                # descriptive stats
print(df.corr(numeric_only=True))    # correlation matrix
# then apply regression / hypothesis tests on the columns of interest
```

Deliverable: a short report with the summary statistics, at least one chart, one regression and one hypothesis test, with your interpretation.

